

Linux :)

study 1편

남궁명수

2026년 6월 10일

제1장. Linux 설치와 시스템 이해	3
1. 가상화 기술과 VMware	3
2. VMware Fusion NAT 네트워크 대역 변경 및 확인	6
3. Hard Disk Interface	10
4. VMware Fusion Nat	12
5. ISO 파일	14
6. macOS에서 ISO 파일 만들기	15
7. systemctl이란?	16
8. systemd란?	18
9. DNF란?	20
10. 자동 업데이트 비활성화 설정이란?	22
11. GNOME Software란?	26
12. /boot/efi란 무엇인가?	28
13. Linux의 최상위 계정 Root란?	32
제2장. Linux 운영 및 관리	34
1. 가상 머신의 네트워크 설정	34
2. NetworkManager와 네트워크 인터페이스 이해	37
3. Linux 설치 미디어와 /dev/sr0의 의미	40
4. Linux 사용자 생성과 /etc/skel의 역할	44
5. 사용자 전환과 파일 권한 관리	48
6. Linux의 링크(Link)와 Inode	54
7. DNF의 동작 방식과 설정 파일	59
8. Rocky Linux DVD를 저장소(Repository)로 사용하는 이유	65
9. DNF 그룹 패키지(Group Package) 관리	70

제1장. Linux 설치와 시스템 이해

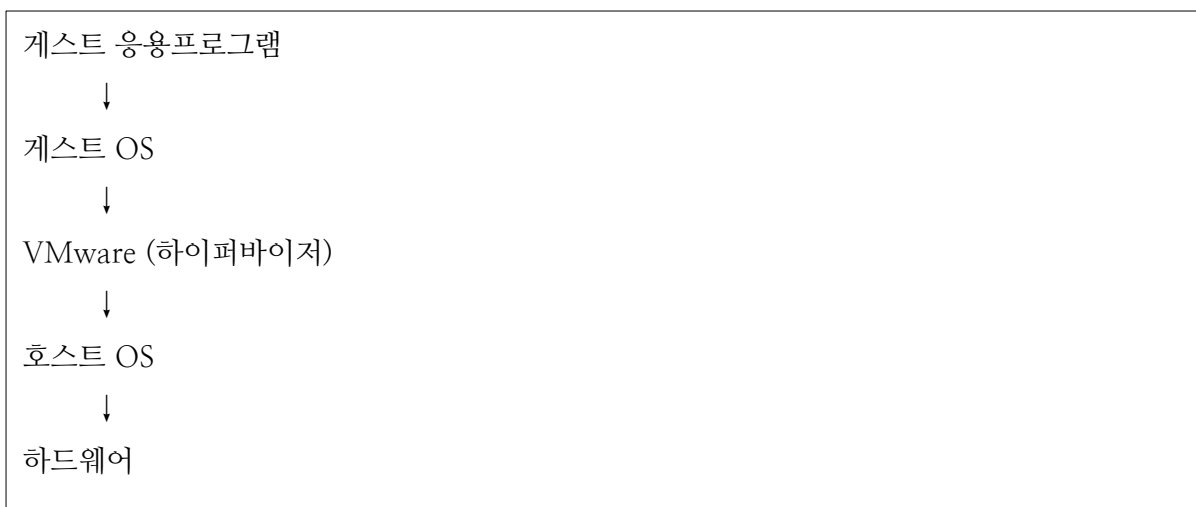
1. 가상화 기술과 VMware

1-1. 가상화(Virtualization)와 하이퍼바이저

현대 컴퓨터에서 VMware와 같은 소프트웨어를 사용할 수 있는 이유는 CPU가 하드웨어 수준의 가상화 기능(Intel VT-x, AMD-V)을 제공하기 때문이다.

이러한 기능을 이용하여 하이퍼바이저(Hypervisor)는 하나의 물리적인 컴퓨터에서 여러 운영체제가 동시에 실행될 수 있도록 자원을 관리한다.

우리가 사용하는 VMware Workstation이나 VMware Fusion은 Type2 하이퍼바이저에 해당하며 구조는 다음과 같다.



Type2 하이퍼바이저는 호스트 운영체제 위에서 실행되는 하나의 응용 프로그램이다.

자체가 운영체제보다 더 아래에 존재하는 것은 아니며, 실제 하드웨어 접근은 호스트 운영체제 통해 이루어진다.

1-2. 호스트 OS와 게스트 OS의 차이

가상화 환경에서 가장 중요한 점은 호스트 OS와 게스트 OS가 하드웨어를 바라보는 방식이 다르다는 것이다.

호스트 OS는 실제 하드웨어를 직접 사용한다. CPU, 메모리, 디스크, 네트워크 카드와 같은 물리 장치를 직접 제어하며 시스템 전체를 관리한다.

반면 게스트 OS는 실제 하드웨어를 직접 볼 수 없다. 대신 VMware가 제공하는 가상 하드웨어를 사용한다.

예를 들면 다음과 같다.

실제 CPU
↓
가상 CPU(vCPU)

실제 RAM
↓
가상 RAM(vRAM)

실제 SSD/HDD
↓
가상 디스크(vDisk)

실제 랜카드
↓
가상 네트워크 어댑터

게스트 OS는 이러한 가상 하드웨어를 실제 장치라고 생각하고 동작한다.

1-3. 가상 하드웨어는 어디서 오는가?

가상 CPU나 가상 메모리가 새롭게 생성되는 것은 아니다.

실제로는 VMware가 물리 하드웨어 자원의 일부를 할당하여 가상 장치처럼 보여주는 것이다.

예를 들어 실제 컴퓨터가 다음과 같은 사양을 가진다고 가정하자.

CPU : 8 Core
RAM : 32GB
SSD : 1TB

그리고 VMware에서 다음과 같이 설정했다.

vCPU : 4 Core
vRAM : 8GB
vDisk : 100GB

그러면 게스트 OS는 다음과 같이 인식한다.

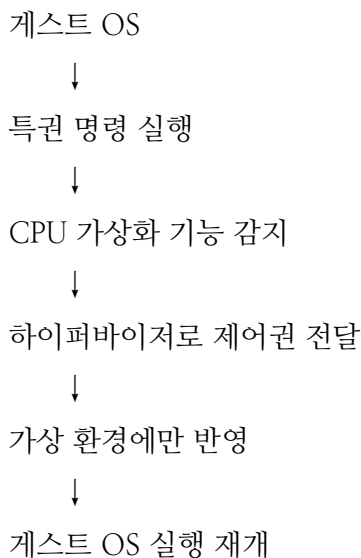
CPU : 4 Core
RAM : 8GB
DISK : 100GB

마치 별도의 컴퓨터에서 실행되는 것처럼 보이지만 실제로는 VMware가 물리 자원의 일부를 나누어 제공한 것이다.

1-4. CPU 가상화 기능의 역할

게스트 OS는 자신이 실제 컴퓨터라고 생각하기 때문에 CPU 제어, 메모리 관리, 인터럽트 처리와 같은 특권 명령을 실행하려고 한다.

하지만 게스트 OS가 실제 하드웨어를 직접 제어하도록 허용하면 시스템 전체가 위험해진다. 그래서 CPU의 가상화 기능(Intel VT-x, AMD-V)이 이를 감지한다.



즉, 게스트 OS는 자신이 CPU를 직접 제어한다고 생각하지만 실제로는 CPU와 하이퍼바이저가 중간에서 이를 관리한다.

1-5. 결론

VMware 환경에서는 호스트 OS가 실제 하드웨어를 직접 사용하고, 게스트 OS는 VMware가 제공하는 가상 하드웨어를 사용한다. 가상 CPU, 가상 메모리, 가상 디스크는 새로운 자원이 아니라 실제 하드웨어 자원의 일부를 VMware가 할당하여 만들어낸 논리적인 장치이다.

또한 CPU의 가상화 기능(Intel VT-x, AMD-V)이 특권 명령을 하이퍼바이저로 전달하고 제어를 중재함으로써, 여러 운영체제가 하나의 물리 컴퓨터에서 동시에 실행될 수 있다.

2. VMware Fusion NAT 네트워크 대역 변경 및 확인

2-1. VMware 네트워크 설정 파일 확인

VMware Fusion은 가상 네트워크 설정 정보를 다음 디렉터리에 저장한다.

```
cd "/Library/Preferences/VMware Fusion/"  
ls -l
```

해당 디렉터리를 확인하면 다음과 같은 파일과 디렉터리를 볼 수 있다.

```
networking  
vmnet1/  
vmnet8/
```

여기서 가장 중요한 파일은 networking 파일이다.

```
cat networking
```

실행 시 다음과 같은 설정을 확인할 수 있다.

```
answer VNET_1_HOSTONLY_SUBNET 192.168.104.0  
  
answer VNET_8_HOSTONLY_SUBNET 192.168.111.0  
answer VNET_8_NAT yes
```

이 파일에는 VMware가 사용하는 가상 네트워크의 IP 대역과 NAT 설정 정보가 저장되어 있다.

2-2. VNET_8의 의미

VMware Fusion에서 일반적으로 사용하는 NAT 네트워크는 vmnet8(VNET_8)이다.

```
VNET_8  
↓  
VMware NAT Network
```

즉, 가상머신이 NAT 방식으로 인터넷을 사용하는 경우 대부분 VNET_8 설정을 사용하게 된다.

설정 파일의 다음 항목은 NAT 네트워크가 사용하는 내부IP 대역을 의미한다.

```
answer VNET_8_HOSTONLY_SUBNET 192.168.111.0
```

예를 들어, 원래 값이

```
answer VNET_8_HOSTONLY_SUBNET 172.16.85.0
```

였다면

```
answer VNET_8_HOSTONLY_SUBNET 192.168.111.0
```

로 변경하는 것은 VMware NAT 네트워크의 내부 주소 대역을 변경한 것이다.

2-3. 왜 IP 대역을 변경하는가?

가상 네트워크가 사용하는 IP 대역이 회사 VPN이나 다른 네트워크와 충돌할 수 있기 때문이다.

예를 들어,

```
회사 VPN  
172.16.0.0/16
```

```
VMware NAT  
172.16.85.0/24
```

와 같이 같은 범위의 주소를 사용하는 경우 라우팅 충돌이 발생할 수 있다.
따라서 사용하지 않는 사설 IP 대역으로 변경한다.

```
192.168.111.0/24
```

이렇게 변경하면 충돌 가능성을 줄일 수 있다.

2-4. VMware 가상 네트워크 인터페이스 확인

설정 변경 후 VMware가 생성한 가상 네트워크 인터페이스를 확인한다.

```
ifconfig -l
```

실행 결과 예시:

```
lo0 gif0 stf0 en0 en1 en2 en3  
...  
bridge100
```

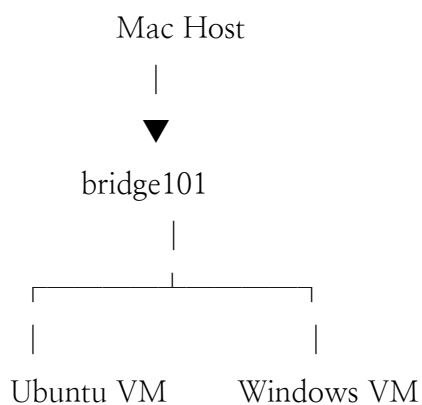
```
bridge101
vmenet0
vmenet1
```

여러 인터페이스가 출력되지만 NAT 네트워크 확인 시에는 bridge101을 확인하면 된다.

2-5. bridge101의 의미

bridge101은 VMware가 생성한 NAT 네트워크용 가상 스위치(Virtual Switch)이다.

구조는 다음과 같다.



모든 가상머신은 이 가상 스위치에 연결된다.

그리고 bridge101은 NAT 기능을 통해 외부 인터넷으로 연결된다.

2-6. bridge101 설정 확인

현재 NAT 네트워크 설정을 확인하려면 다음 명령을 사용한다.

```
ifconfig bridge101
```

예시 출력

```
bridge101: flags=...
inet 192.168.111.1
netmask 0xfffff00
broadcast 192.168.111.255
```

여기서 192.168.111.1은 VMware NAT 네트워크의 게이트웨이 역할을 수행하는 주소이다.

즉, 현재 구조는 다음과 같다.

Mac Host

|

bridge101

|

192.168.111.1

|

192.168.111.0/24

|

Ubuntu VM

192.168.111.x

2-7. 변경 결과 확인

설정 변경이 정상적으로 적용되었다면 가상머신은 DHCP를 통해 새로운 IP 주소를 받게 된다.

예를 들어 이전에는

172.16.85.128

을 사용했다면,

변경 후에는

192.168.111.128

과 같은 주소를 할당받게 된다.

Linux VM에서 다음 명령으로 확인할 수 있다.

ip addr 또는 hostname -I

DHCP(Dynamic Host Configuration Protocol)

기기가 네트워크에 연결될 때 인터넷 통신에 필요한 IP 주소 및 관련 설정(서브넷 마스크, 게이트웨이, DNS)을 자동으로 할당해주는 통신 규약

3. Hard Disk Interface

3-1. SATA(Serial ATA)

SATA는 오랫동안 개인용 컴퓨터에서 가장 널리 사용된 저장장치 인터페이스이다.

원래는 HDD를 연결하기 위해 설계되었으며, 이후 SSD도 동일한 인터페이스를 사용하게 되었다. 구조가 단순하고 대부분의 운영체제에서 기본적으로 지원하기 때문에 호환성이 매우 뛰어나다. 다만 SATA는 HDD 시대의 설계를 기반으로 만들어졌기 때문에 SSD의 성능을 완전히 활용하기에는 한계가 있으며, 일반적으로 최대 전송 속도는 약 600MB/s 수준이다.

VMware에서 SATA를 선택하면 게스트 OS는 SATA 방식의 저장장치가 연결된 것으로 인식한다.

3-2. SCSI(Small Computer System Interface)

SCSI는 서버와 워크스테이션 환경을 위해 발전한 저장장치 인터페이스이다.

여러 저장장치를 효율적으로 관리하고 동시에 많은 입출력 작업을 처리하는 데 강점을 가지고 있어 기업용 서버 환경에서 널리 사용되었다.

현재 SAS(Serial Attached SCSI) 형태로 발전하여 서버 시장에서 활용되고 있다.

VMware에서는 SCSI 컨트롤러를 기본으로 사용하는 경우가 많으며, 특히 데이터베이스 서버나 파일 서버와 같은 디스크 입출력이 많은 가상머신에서 자주 사용된다. 게스트 OS는 실제 하드웨어와 관계없이 SCSI 방식의 디스크가 연결된 것으로 인식하게 된다.

3-3. NVMe(Non-Volatile Memory Express)

NVMe는 SSD를 위해 새롭게 설계된 최신 저장장치 인터페이스이다.

기존 SATA가 HDD 환경을 기준으로 설계된 것과 달리, NVMe는 SSD의 높은 성능을 최대한 활용할 수 있도록 PCIe 버스를 직접 사용한다.

이로 인해 지연 시간이 매우 낮고 초당 처리 가능한 입출력 작업(IOPS)이 크게 향상되며, 전송 속도 역시 SATA보다 훨씬 빠르다.

현대의 대부분의 고성능 SSD가 NVMe 방식을 사용하고 있으며, VMware에서 NVMe를 선택하면 게스트 OS는 최신 NVMe SSD가 장착된 컴퓨터를 사용하는 것처럼 인식한다.

최신 운영체제를 사용하는 일반적인 가상머신 환경에서는 NVMe를 사용하는 것이 가장 자연스럽고 성능 측면에서도 유리하다.

3-4. VMware에서의 의미

VMware에서 SATA, SCSI, NVMe를 선택하는 것은 실제 디스크의 종류를 결정하는 것이 아니라, 게스트 OS에게 어떤 저장장치를 컨트롤러를 제공할 것인지를 결정하는 것이다.

즉, 실제 컴퓨터에 NVMe SSD가 장착되어 있더라도 게스트 OS에게는 SATA 디스크로 보이게 할 수 있고, 반대로 실제 저장장치가 SATA SSD여도 게스트 OS에게는 NVMe SSD가 연결된 것처럼 보이게 할 수 있다.

VMware는 중간에서 가상 컨트롤러를 제공하고 실제 저장장치와의 통신을 대신 처리하기 때문에, 게스트 OS는 자신에게 제공된 가상 하드웨어만 인식하게 된다.

따라서 일반적인 최신 운영체제에서는 NVMe, 서버 환경에서는 SCSI, 오래된 운영체제와의 호환성이 필요한 경우에는 SATA를 선택하는 경우가 많다.

4. VMware Fusion Nat

4-1. 원래 상태

VMware Fusion은 NAT 네트워크를 제공하기 위해 자체적인 가상 네트워크를 생성한다. 이 가상 네트워크는 일반적으로 vmnet8이라는 이름으로 관리되며, 내부적으로 별도의 IP 대역을 사용한다. 당시 VMware는 172.16.85.0/24 대역을 사용하고 있었던 것으로 보인다. 따라서 VMware가 생성한 가상 네트워크 안에서는 172.16..85.1이 게이트웨이 역할을 하고, 가상머신들은 172.16..85.2 ~ 172.16..85.254 범위의 IP 주소를 DHCP를 통해 할당받아 사용하고 있었을 가능성이 높다. 즉, VMware 내부에는 이미 하나의 독립된 사설 네트워크가 구성되어 있었던 것이다.

4-2. networking 파일 수정

/Library/Preferences/VMware Fusion/networking 파일은 VMware의 가상 네트워크 설정을 저장하는 구성 파일이다. 여기서 answer VNET_8_HOSTONLY_SUBNET 172.16.85.0 값을 192.168.111.0으로 변경한 것은 VMware가 사용하는 내부 네트워크 대역 자체를 변경한 것이다. 쉽게 말해 기존에는 VMware가 172.16.85.x 네트워크를 사용하도록 설정되어 있었는데, 이를 192.168.111.x 네트워크를 사용하도록 재구성한 것이다.

이 작업은 실제 인터넷 설정을 바꾸는 것이 아니라 VMware 내부에서 VM들이 사용하는 사설 네트워크 주소 체계를 변경하는 작업이다.

4-3. bridge101 상태

설정 변경 후 ifconfig bridge101 결과를 확인했을 때 192.168.111.1 주소가 표시되었다.

이는 VMware가 변경된 설정을 적용하여 새로운 네트워크를 생성했다는 의미이다.

여기서 192.168.111.1은 VMware 가상 네트워크의 게이트웨이 역할을 수행한다.

따라서 현재 VMware 네트워크 구조는 Mac 호스트가 bridge101 인터페이스를 통해 192.168.111.1 주소를 가지고 있고, VMware 내부의 가상머신들은 동일한 192.168.111.0/24 네트워크 대역에 속하게 된다.

4-4. bridge101의 역할

bridge101은 VMware가 생성한 가상 네트워크 장치로, 네트워크 관점에서 보면 일종의 가상 스위치(Virtual Switch)라고 이해할 수 있다.

실제 네트워크 환경에서 여러 컴퓨터가 스위치에 연결되는 것처럼, VMware 내부의 여러 가상머신도 모두 bridge101에 연결된다.

예를 들어, Ubuntu VM, Windows VM, 기타 VM 들이 존재한다면 모두 이 가상 스위치를 통해 서로 통신하게 된다. 또한, bridge101은 VMware의 NAT 기능과 연결되어 있어 VM들이 외부 인터넷에 접근할 수 있도록 중계 역할도 수행한다.

4-5. 네트워크 대역을 변경한 이유

이러한 변경은 대부분 네트워크 충돌을 방지하기 위해 수행된다. 예를 들어, 회사 VPN이나 특정 네트워크 환경에서 이미 172.16.0.0/16 대역을 사용하고 있다면, VMware가 사용하는 172.16.85.0/24 대역과 경로가 겹쳐 문제가 발생할 수 있다.

운영체제는 어느 네트워크로 패킷을 보내야 하는지 혼란을 겪게 되고, 그 결과 VPN 접속 오류나 통신 불가 현상이 발생할 수 있다. 따라서 사용하지 않는 별도의 대역인 192.168.111.0/24로 변경하여 이러한 충돌 간섭성을 제거한 것이다.

4-6. 변경 후 기대되는 결과

설정이 정상적으로 적용되면 VMware 내부의 DHCP 서버도 새로운 네트워크 대역을 기준으로 IP 주소를 할당하게 된다. 따라서 Ubuntu VM이 DHCP를 통해 새로운 주소를 받을 경우 기존의 172.16.85.xxx 대신 192.168.111.xxx 형태의 IP 주소를 갖게 된다.

예를 들어, 기존에 172.16.85.128을 사용하던 VM은 DHCP 갱신 후 192.168.111.128과 같은 주소를 받게 될 수 있다. 이는 VMware 내부 네트워크가 완전히 새로운 주소 체계로 동작하기 시작했다는 것을 의미한다.

5. ISO 파일

ISO 파일은 CD, DVD, Blu-ray와 같은 저장 매체의 내용을 그대로 하나의 파일로 저장한 디스크 이미지 파일이다. 단순히 여러 파일을 압축한 것이 아니라 디스크의 파일 구조, 부팅 정보, 설치 프로그램 등을 포함하여 디스크 전체를 복제한 형태라고 볼 수 있다.

따라서 ISO 파일은 원본 디스크와 거의 동일한 역할을 수행한다.

과거에는 운영체제를 설치하기 위해 실제 DVD를 컴퓨터에 삽입해야 했지만, 현재는 운영체제 제작사들이 설치 이미지를 ISO 파일 형태로 배포한다.

사용자는 해당 ISO 파일을 다운로드한 뒤 USB에 기록하거나 가상머신에 연결하여 운영체제를 설치할 수 있다. 이 방식은 물리적인 설치 미디어가 필요 없고 배포와 보관이 편리하다는 장점이 있다.

VMware와 같은 가상화 환경에서는 ISO 파일이 특히 중요하다. 가상머신은 실제 DVD 드라이브가 없기 때문에 VMware가 ISO 파일을 가상 CD/DVD 장치로 연결해 준다.

그러면 게스트 운영체제가 마치 실제 설치 DVD가 삽입된 것처럼 인식하고 해당 ISO 파일로부터 부팅하여 운영체제를 설치할 수 있다.

ISO 파일은 ZIP 파일과는 성격이 다르다. ZIP 파일은 단순히 파일을 압축하여 저장하는 것이 목적이지만, ISO 파일은 디스크 자체를 이미지 형태로 저장하는 것이 목적이다.

따라서 ISO 파일은 부팅이 가능하며 운영체제 설치 미디어로 사용할 수 있다는 점에서 일반 압축 파일과 구별된다.

결론적으로 ISO 파일은 운영체제 설치 디스크를 하나의 파일 형태로 보관한 디스크 이미지 파일이며, 실제 CD나 DVD를 대체하는 현대적인 설치 매체라고 이해할 수 있다.

6. macOS에서 ISO 파일 만들기

macOS에서는 디스크 유틸리티(Disk Utility)와 터미널 명령어(hdituil)를 이용하여 ISO 파일을 생성할 수 있다. ISO 파일은 CD, DVD와 같은 디스크의 내용을 하나의 이미지 파일로 저장하는 형식으로, 운영체제 설치 이미지나 데이터 백업, 가상머신 설치용 이미지 제작 등에 널리 사용된다.

가장 일반적인 방법은 디스크 유틸리티를 사용하는 것이다. 디스크 유틸리티를 실행한 후 [파일] -> [새로운 이미지 만들기] -> [폴더의 이미지] 또는 [빈 이미지] 메뉴를 선택하여 이미지 파일을 생성할 수 있다. 생성된 파일은 일반적으로 .cdr 형식으로 저장되는데, 이는 macOS에서 사용하는 디스크 이미지 형식이다.

VMware나 VirtualBox와 같은 가상화 환경에서는 보통 .iso 형식을 사용하기 때문에 .cdr 파일을 ISO 파일로 변환하는 과정이 필요하다 macOS에서는 hdiutil 명령어를 이용하여 이를 수행할 수 있다.

```
hdiutil makehybrid -iso -joliet -o Name.iso Name.cdr
```

위 명령어에서 Name.cdr은 원본 디스크 이미지 파일이고, Name.iso는 생성될 ISO 파일의 이름이다. -iso 옵션은 ISO 9660 형식으로 변환하도록 지정함, -joliet 옵션은 Windows 환경에서 긴 파일 이름과 Unicode 문자 자원을 가능하게 한다.

변환이 완료되면 Name.iso 파일이 생성되며, 이 파일은 VMware Fusion, VirtualBox, 기타 가상 머신 소프트웨어에서 설치 미디어로 사용할 수 있다.

또한, USB 부팅 디스크를 만들거나 다른 운영체제로 전송하여 사용할 수도 있다.

결론적으로 macOS에서 ISO파일을 만들기 위해서는 먼저 디스크 유틸리티를 이용하여 디스크 이미지를 생성한 뒤, hdituil makehybrid 명령어를 통해 .cdr 파일을 ISO 형식으로 변환하는 과정을 거치면 된다. 이는 macOS에서 가장 표준적인 ISO 생성 방법 중 하나이다.

7. systemctl이란?

systemctl은 Linux에서 시스템 서비스(Service)를 관리하기 위한 명령어이다.

현대의 대부분의 Linux 배포판은 systemd라는 초기화 시스템(init system)을 사용하며, systemctl은 이 systemd를 제어하는 대표적인 도구이다.

과거 Linux에서는 서비스 관리를 위해 SysVinit과 같은 방식이 사용되었지만, 현재는 대부분의 배포판이 systemd를 채택하고 있다. 따라서 사용자는 systemctl 명령어를 통해 웹서버, 데이터베이스 서버, SSH 서버 등 다양한 서비스를 시작하거나 중지하고 상태를 확인할 수 있다.

예를 들어, 웹 서버가 실행 중인지 확인하려면 다음과 같은 명령을 사용할 수 있다.

```
systemctl status nginx
```

서비스를 시작하거나 중지할 때는 다음과 같이 사용한다.

```
sudo systemctl start nginx  
sudo systemctl stop nginx
```

서비스를 재시작하거나 설정 변경 후 다시 적용해야 할 경우에는 다음 명령을 사용한다.

```
sudo systemctl restart nginx
```

또한 Linux가 부팅될 때 특정 서비스가 자동으로 실행되도록 설정할 수도 있다.

```
sudo systemctl enable nginx
```

반대로 부팅 시 자동 실행을 해제하려면 다음과 같이 입력한다.

```
sudo systemctl disable nginx
```

systemctl은 단순히 서비스를 시작하고 중지하는 역할만 하는 것이 아니다.

실제로는 systemd와 통신하여 서비스의 상태를 관리하고, 서비스 간의 의존성을 처리하며, 시스템 부팅 과정에서 어떤 서비스가 언제 실행될지 결정하는 역할까지 담당한다.

즉, systemctl을 통해 systemd에게 명령을 전달하고, systemd가 실제 서비스 관리를 수행하는 구조이다.

VMware나 실제 서버 환경에서 Ubuntu를 사용할 때도 대부분의 서버 프로그램은 systemd 서비스 형태로 동작한다. 예를 들어, SSH 서버, Docker Nginx, MYSQL 등의 프로그램은 모두 서비스로 등록되어 있으며, 관리자는 systemctl 명령어를 이용하여 실제 실행 상태를 확인하고 운영하게 된다.

결론적으로 systemctl은 Linux 서비스 관리 도구이며, systemd를 제어하여 서비스의 시작, 중지, 재시작, 상태 확인 및 부팅 시 자동 실행 설정 등을 수행하는 명령어라고 이해하면 된다. 쉽게 말해 Linux에서 실행 중인 서버 프로그램들을 관리하는 중앙 제어 도구라고 볼 수 있다.

8. systemd란?

systemd는 Linux 운영체제가 부팅된 이후 시스템 전체를 관리하는 시스템 및 서비스 관리자(System and Service Manager)이다. 쉽게 말하면, Linux가 켜진 후 가장 먼저 실행되는 핵심 프로그램 중 하나로, 시스템을 정상적인 사용 상태까지 올리고 필요한 서비스들을 관리하는 역할을 담당한다.

컴퓨터의 전원을 켜면 커널(Kernel)이 먼저 메모리에 로드된다. 커널은 하드웨어를 초기화한 뒤 가장 먼저 실행할 사용자 공간(User Space)의 프로그램을 찾는데, 현대 Linux에서는 대부분 systemd가 이 역할을 수행한다. 따라서 systemd는 커널이 실행한 첫 번째 사용자 공간 프로세스이며 일반적으로 PID(Process ID) 1번을 가진다.

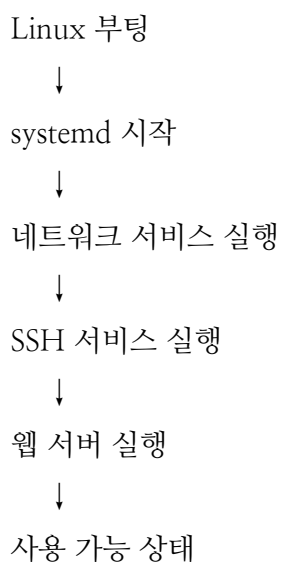
systemd의 주요 역할을 시스템 부팅 과정에서 필요한 서비스들을 시작하는 것이다.

예를 들어, 네트워크 서비스, SSH 서버, 웹 서버, 데이터베이스 서버와 같은 프로그램들은 운영체제가 부팅된 이후 자동으로 실행되어야 한다.

systemd는 이러한 서비스들의 실행 순서를 관리하고, 서로 의존성이 있는 경우 적절한 순서대로 시작하도록 제어한다.

예를 들어, 웹 서버가 정상적으로 동작하려면 먼저 네트워크가 활성화되어 있어야 한다.

systemd는 이러한 의존 관계를 인식하여 네트워크 서비스를 먼저 시작한 뒤 웹 서버를 실행한다.

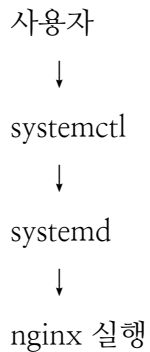


또한 systemd는 단순히 서비스를 시작하는 것에 그치지 않는다. 실행 중인 서비스를 지속적으로 감시하며, 서비스가 비정상적으로 종료될 경우 자동으로 재시작하도록 설정할 수도 있다. 따라서, 서버 환경에서는 시스템의 안정성과 가용성을 높이는 중요한 역할을 수행한다.

사용자가 자주 사용하는 systemctl 명령어도 실제로는 systemd를 제어하기 위한 도구이다.
예를 들어,

```
sudo systemctl start nginx
```

명령을 실행하면,



```
graph TD; A[사용자] --> B[systemctl]; B --> C[systemd]; C --> D[nginx 실행];
```

과 같은 과정이 이루어진다.

즉, systemctl은 명령을 전달하는 도구이고, 실제로 서비스를 관리하는 주체는 systemd이다. 현대의 Ubuntu, Debian, Fedora, CentOS, Rocky Linux 등 대부분의 Linux 배포판은 systemd를 기본 초기화 시스템으로 사용한다. 따라서 서버를 운영하거나 Docker, Nginx, MySQL, SSH와 같은 서비스를 관리할 때 systemd에 대한 이해는 필수적이다.

결론적으로 systemd는 Linux가 부팅된 이후 시스템 전체를 관리하는 핵심 관리자이며, 서비스의 시작, 중지, 재시작, 자동 실행 및 의존성 관리를 담당하는 Linux의 중앙 관리 시스템이라고 이해하면 된다. 쉽게 말해 Linux 서버의 “운영 총괄 관리자” 역할을 수행하는 프로그램이다.

9. DNF란?

DNF(Dandified YUM)는 Linux에서 소프트웨어 패키지를 설치, 업데이트, 삭제 및 관리하기 위한 패키지 관리자(Package Manager)이다.

주로 Fedora, Rocky Linux, AlmaLinux, RHEL(Red Hat Enterprise Linux) 계열 배포판에서 사용되며, 과거에 사용되던 YUM(Yellowdog Updater Modified)을 개선한 후속 버전이다.

운영체제를 설치한 직후에는 필요한 프로그램들이 모두 설치되어 있지 않다.

예를 들어, 웹 서버를 운영하기 위해 Nginx를 설치하거나, 데이터베이스 서버를 위해 MariaDB를 설치해야 할 수 있다. 이러한 프로그램들을 인터넷 저장소에서 다운로드하고 설치하는 작업을 DNF가 담당한다.

예를 들어, Nginx를 설치하려면 다음과 같은 명령을 사용한다.

```
sudo dnf install nginx
```

이 명령을 실행하면 DNF는 저장소에 접속하여 필요한 패키지를 찾고, 관련 의존성 패키지까지 함께 다운로드하여 설치한다. 사용자는 복잡한 파일 다운로드나 설정 과정을 직접 처리할 필요가 없다.

DNF는 단순히 프로그램을 설치하는 것뿐만 아니라 시스템 전체를 최신 상태로 유지하는 역할도 수행한다. 다음 명령을 실행하면 설치된 패키지들의 업데이트 여부를 확인하고 최신 버전으로 갱신할 수 있다.

```
sudo dnf update
```

또한 더 이상 필요하지 않은 패키지를 제거할 수도 있다.

```
sudo dnf remove nginx
```

DNF의 중요한 특징 중 하나는 의존성(Dependency)관리이다. 많은 프로그램들은 다른 라이브러리나 패키지를 필요로 한다. 사용자가 특정 프로그램을 설치하면 DNF는 필요한 의존성을 자동으로 분석하여 함께 설치한다. 반대로 프로그램을 제거할 때도 불필요한 패키지를 정리할 수 있다.

내부적으로 보면 DNF는 다음과 같은 구조로 동작한다.



여기서 저장소는 Linux 배포판에서 제공하는 공식 소프트웨어 서버라고 생각하면 된다. DNF는 이 저장소와 통신하여 필요한 패키지를 검색하고 설치한다.

Ubuntu 계열에서 사용하는 apt와 비교하면 역할은 거의 동일하다.

Ubuntu/Debian → apt
Fedora/RHEL/Rocky/AlmaLinux → dnf

즉, 운영체제 계열에 따라 사용하는 패키지 관리 도구만 다를 뿐, 프로그램 설치와 관리라는 목적은 같다.

결론적으로, DNF는 Fedora 및 RHEL 계열 Linux에서 사용하는 패키지 관리 도구로, 저장소와 연동하여 소프트웨어 설치, 업데이트, 삭제 및 의존성 관리를 수행하는 시스템이라고 이해하면 된다. 쉽게 말해 Windows의 앱 스토어나 macOS의 패키지 관리 도구처럼 Linux에서 프로그램을 관리하는 공식적인 방법이라고 볼 수 있다.

10. 자동 업데이트 비활성화 설정이란?

다음 명령들은 Fedora, Rocky Linux, AlmaLinux 등 GNOME 환경과 DNF 패키지 관리자를 사용하는 Linux에서 자동 업데이트 관련 기능을 비활성화하는 설정이다.

```
gsettings set org.gnome.software download-updates false
gsettings set org.gnome.software allow-updates false
systemctl disable dnf-makecache.timer
```

이 설정을 적용하면 운영체제가 백그라운드에서 자동으로 업데이트를 확인하거나 다운로드하는 동작을 최소화할 수 있다. 서버 실습 환경이나 VMware 가상머신에서 불필요한 네트워크 사용을 줄이고 싶을 때 자주 사용된다.

10-1. GNOME Software 자동 다운로드 비활성화

첫 번째 명령은 GNOME Software의 자동 다운로드 기능을 끄는 설정이다.

```
gsettings set org.gnome.software download-updates false
```

GNOME Software는 Ubuntu의 App Center와 비슷한 역할을 하는 그래픽 패키지 관리자이다. 기본적으로는 새로운 업데이트가 발견되면 백그라운드에서 패키지를 자동으로 다운로드할 수 있다.

업데이트 발견



자동 다운로드



사용자에게 설치 알림

하지만 위 명령을 실행하면

업데이트 발견



다운로드 안 함



사용자가 직접 수행

형태로 변경된다.

즉, 업데이트 파일을 미리 받아두지 않는다.

10-2. GNOME Software 업데이트 허용 비활성화

두 번째 명령은 GNOME Software의 업데이트 기능 자체를 비활성화한다.

```
gsettings set org.gnome.software allow-updates false
```

이 설정을 적용하면 GNOME Software는 업데이트 기능을 제공하지 않거나 업데이트 알림을 표시하지 않게 된다.

즉 사용자는 GUI 환경에서

GNOME Software



업데이트 확인



업데이트 설치

과정을 수행할 수 없게 된다.

대신 관리자는 직접 터미널에 DNF 명령어를 사용해야 한다.

```
sudo dnf update
```

10-3. dnf-makecache.timer 비활성화

세 번째 명령은 systemd 타이머를 비활성화한다는 설정이다.

```
systemctl disable dnf-makecache.timer
```

DNF는 기본적으로 일정 시간마다 저장소 정보를 갱신하기 위해 캐시를 생성한다.

동작 과정은 다음과 같다.

정해진 시간 도달



dnf-makecache.timer 실행



저장소 서버 접속



패키지 목록 갱신



캐시 업데이트

이 기능 덕분에 사용자가

```
sudo dnf install nginx
```

같은 명령을 실행했을 때 최신 패키지 정보를 빠르게 확인할 수 있다.

하지만 실습용 VM이나 인터넷 사용을 최소화해야 하는 환경에서는 불필요한 네트워크 트래픽이 발생할 수 있다.

따라서 타이머를 비활성화하면

자동 캐시 갱신 중지



필요할 때만 dnf 실행

방식으로 변경된다.

10-4. 설정 후 시스템 변화

위 세 가지 설정을 모두 적용하면 시스템은 자동으로 업데이트를 다운로드하지 않고, GNOME Software를 통한 업데이트도 비활성화되며, DNF의 주기적인 저장소 캐시 갱신도 수행하지 않게 된다.

즉, 기존에는

업데이트 확인



자동 다운로드



캐시 갱신



업데이트 알림

과정이 자동으로 진행되었다면,

설정 이후에는,

관리자가 직접 명령 실행




```
sudo dnf update
```

↓

필요한 경우에만 업데이트

방식으로 변경된다.

결론적으로 이 명령들은 Linux의 자동 업데이트 및 자동 저장소 동기화 기능을 비활성화하여 시스템을 수동 관리 방식으로 전환하는 설정이라고 이해하면 된다.

이는 VMware 실습 환경, 교육용 VM, 네트워크 사용을 최소화해야 하는 서버 환경에서 자주 사용된다.

11. GNOME Software란?

GNOME Software는 GNOME 데스크톱 환경에서 제공하는 그래픽 기반 소프트웨어 관리 프로그램이다. 사용자는 터미널 명령어를 입력하지 않고도 프로그램을 설치, 제거, 업데이트 할 수 있도록 만들어진 일종의 “앱 스토어” 역할을 한다.

Linux에서 프로그램을 설치하는 방법은 크게 두 가지가 있다. 하나는 dnf, apt와 같은 패키지 관리자를 터미널에서 사용하는 방법이고, 다른 하나는 GNOME Software와 같은 그래픽 인터페이스(GUI)를 사용하는 방법이다. GNOME Software는 내부적으로 패키지 관리자와 통신하지만, 사용자는 복잡한 명령어를 기억할 필요 없이 마우스 클릭만으로 프로그램을 관리할 수 있다.

예를 들어, 사용자가 웹 브라우저나 개발 도구를 설치하려고 할 때, 터미널에서는 다음과 같은 명령을 입력해야 한다.

```
sudo dnf install firefox
```

하지만 GNOME Software에서는 프로그램 이름을 검색한 후 설치 버튼만 누르면 된다. 실제 설치 작업은 내부적으로 DNF나 PackageKit같은 패키지 관리 시스템이 수행한다.

GNOME Software는 단순히 프로그램 설치 기능만 제공하는 것이 아니다. 시스템에 설치된 프로그램 목록을 관리하고, 새로운 버전이 존재하는지 확인하며, 운영체제 업데이트와 보안 패치도 안내한다. 따라서 사용자는 별도의 명령어를 사용하지 않고도 시스템을 최신 상태로 유지할 수 있다.

구조를 단순하게 표현하면 다음과 같다.

```
사용자
  ↓
GNOME Software
  ↓
PackageKit
  ↓
DNF / APT
  ↓
저장소(Repository)
```

즉, GNOME Software는 실제 패키지 관리자가 아니라 사용자가 편하게 사용할 수 있도록 만든 그래픽 인터페이스라고 볼 수 있다.

앞서 사용한 다음 명령어들도 GNOME Software와 관련된 설정이다.

```
gsettings set org.gnome.software download-updates false
gsettings set org.gnome.software allow-updates false
```

첫 번째 명령은 GNOME Software가 업데이트를 자동으로 다운로드하지 못하도록 설정하고, 두 번째 명령은 GNOME Software를 통한 업데이트 기능 자체를 비활성화하는 역할을 한다.

결론적으로 GNOME Software는 Linux에서 프로그램 설치, 제거, 업데이트를 그래픽 환경에서 쉽게 수행할 수 있도록 제공하는 소프트웨어 관리 도구이다.

Windows의 Microsoft Store나 macOS의 App Store와 비슷한 역할을 하지만, 실제 설치 작업은 내부의 패키지 관리자(DNF, APT 등)가 수행한다.

따라서 GNOME Software는 패키지 관리 시스템을 사용자 친화적으로 사용할 수 있게 해주는 GUI 프론트엔드라고 이해하면 된다.

12. /boot/efi란 무엇인가?

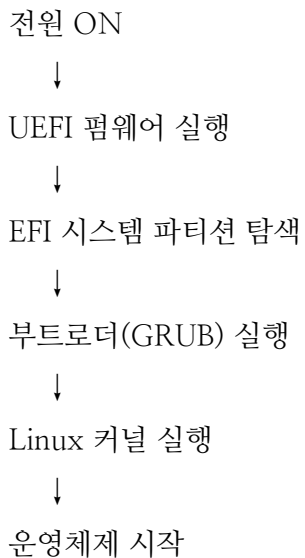
/boot/efi는 Linux에서 UEFI(Unified Extensible Firmware Interface) 부팅에 필요한 파일들이 저장되는 마운트 지점(Mount Point)이다.

운영체제가 설치될 때 UEFI 방식으로 부팅하기 위해 반드시 필요한 EFI 시스템 파티션(ESP, EFI System Partition)이 이 위치에 연결된다.

과거 BIOS 방식에서는 부팅에 필요한 정보가 디스크의 MBR(Master Boot Record)에 저장되었다. 하지만 현대 컴퓨터는 대부분 UEFI 방식을 사용하며, UEFI는 디스크 안에 별도의 EFI 시스템 파티션을 만들어 그 안에 부팅 파일을 저장한다.

Linux에서는 이 EFI 시스템 파티션을 일반적으로 /boot/efi 디렉터리에 마운트하여 사용한다.

부팅 과정은 대략 다음과 같이 이루어진다.



즉, /boot/efi는 UEFI가 가장 먼저 접근하는 부팅 파일들이 저장된 장소라고 볼 수 있다.

12-1. EFI 시스템 파티션의 역할

수동 파티션 설정을 할 때 /boot/efi로 지정하는 파티션은 일반적인 데이터 저장 공간이 아니다. 이 파티션에는 운영체제를 부팅하기 위한 파일들이 저장된다.

예를 들어, 내부에는 다음과 같은 파일들이 존재할 수 있다.

```
EFI/  
├── ubuntu/  
│   ├── grubx64.efi  
│   └── shimx64.efi  
├── fedora/  
├── Microsoft/  
└── Boot/
```

여기서 .efi 파일들은 운영체제를 시작시키는 부트로더 역할을 한다.

12-2. 수동 파티션 설정 시 /boot/efi를 만드는 이유

Linux 설치 과정에서 “수동 파티션”을 선택하면 사용자가 직접 파티션의 용도를 지정해야 한다.

이때 UEFI 방식으로 설치하려면 EFI 시스템 파티션을 생성하고 마운트 지점을 /boot/efi로 지정해야 한다.

일반적인 설정 예시는 다음과 같다.

마운트 지점	크기	파일 시스템
/boot/efi	300~600MB	FAT32
/	나머지 공간	ext4, xfs 등

여기서 /boot/efi는 FAT32 파일 시스템을 사용해야 한다.

이는 UEFI 펌웨어가 FAT32를 읽을 수 있도록 설계되어 있기 때문이다.

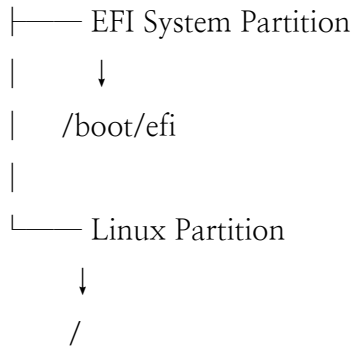
12-3. VMware에서의 의미

VMware, VirtualBox, Hyper-V같은 가상 머신도 UEFI 모드를 지원한다.

만약 VM을 UEFI 방식으로 생성했다면 Linux 설치 시에도 실제 컴퓨터와 동일하게 EFI 시스템 파티션이 필요하다.

예를 들어, Fedora나 Ubuntu를 UEFI 모드로 설치할 경우:

가상 디스크



구조가 생성된다.

설치 프로그램은 /boot/efi에 GRUB 부트로더를 설치하고, 이후 VM이 켜질 때 UEFI가 해당 파일을 읽어 Linux를 부팅한다.

12-4. /boot와 /boot/efi의 차이

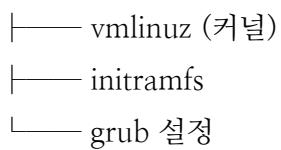
많은 사람들이 /boot와 /boot/efi를 혼동한다.

/boot는 Linux 커널과 초기 RAM 디스크(initramfs) 같은 부팅 관련 파일을 저장하는 일반 디렉터리이다.

반면, /boot/efi는 UEFI가 직접 읽는 EFI 시스템 파티션이다.

즉, 역할이 다르다.

/boot



/boot/efi



12-5. 결론

/boot/efi는 UEFI 방식으로 Linux를 부팅하기 위해 필요한 EFI 시스템 파티션의 마운트 지점이다. 이 파티션에는 UEFI가 직접 실행하는 부트로더 파일이 저장되며, 수동 파티션 설정 시에는 일반적으로 FAT32파일 시스템으로 300~600MB 정도의 크기를 할당한다. 현대의 대부분의 물리 서버, PC, 그리고 VMware 가상머신은 UEFI 방식을 사용하므로, Linux를 설치할 때 /boot/efi 파티션은 사실상 필수적인 부팅 영역이라고 이해하면 된다.

13. Linux의 최상위 계정 Root란?

root는 Linux 시스템에서 모든 권한을 가진 최고 관리자 계정(Superuser)이다.

Windows의 Administrator 계정과 비슷한 역할을 하지만, Linux에서는 훨씬 강력한 권한을 가진다. 일반 사용자는 시스템이 허용한 범위 내에서만 작업할 수 있지만, root는 시스템 전체에 대한 완전한 제어 권한을 가진다.

Linux에서는 파일, 디렉터리, 프로세스, 네트워크 설정, 사용자 계정 등 모든 자원에 접근 권한이 설정되어 있다. 일반 사용자는 자신의 파일이나 허가된 영역만 수정할 수 있지만, root는 이러한 권한 제한을 사실상 받지 않는다. 따라서 시스템 설정 변경, 소프트웨어 설치 및 제거, 사용자 계정 관리, 서비스 제어와 같은 관리 작업은 대부분 root 권한이 필요하다.

예를 들어, 새로운 프로그램을 설치하거나 시스템 서비스를 관리할 때 다음과 같은 명령을 사용한다.

```
sudo dnf install nginx
sudo systemctl restart sshd
```

여기서 sudo는 일반 사용자가 일시적으로 root 권한을 빌려 명령을 실행하도록 해주는 도구이다. 실제로 명령은 root 권한으로 실행된다.

Linux 내부적으로 root 계정은 특별한 사용자 ID(UID)를 가진다.

일반 사용자는 UID가 1000 이상인 경우가 많지만, root는 항상 UID가 0이다. Linux 커널은 UID가 0인 사용자를 특별하게 취급하며, 권한 검사 과정에서 거의 모든 제한을 우회할 수 있도록 허용한다.

다만 root 권한은 매우 강력한 만큼 위험하기도 하다. 일반 사용자가 실수로 파일 하나를 삭제하면 자신의 영역에만 영향을 미치지만, root가 잘못된 명령을 실행하면 운영체제 전체가 손상될 수 있다. 예를 들어, 다음과 같은 명령은 시스템 전체 파일을 삭제하는 매우 위험한 명령이다.

```
rm -rf /
```

일반 사용자라면 권한 부족으로 실행되지 않지만, root는 이를 실제로 수행할 수 있다.

그래서 현대 Linux 배포판들은 보안을 위해 평소에는 일반 사용자 계정을 사용하고, 필요한 순간에만 sudo를 통해 root 권한을 얻는 방식을 권장한다.

또한, root 계정의 홈 디렉터리는 일반 사용자의 /home/사용자명과 다르게 /root이다.

```
/
|
|—— home
|
|   |—— user
|   |
|   |—— root
```

여기서 /root는 root 사용자의 개인 작업 공간이며, 시스템 최상위 디렉터리인 /와는 다른 개념이다.

결론적으로 root는 Linux 시스템의 최상위 관리자 계정으로, 모든 파일과 설정에 접근하고 변경할 수 있는 절대적인 권한을 가진 사용자이다.

시스템 관리 작업의 대부분은 root 권한이 필요하며, 보안과 안정성을 위해 평소에는 일반 사용자 계정을 사용하고 필요한 경우에만 sudo를 통해 root권한을 사용하는 것이 일반적인 운영 방식이다.

제2장. Linux 운영 및 관리

1. 가상 머신의 네트워크 설정

가상머신이 네트워크를 통해 다른 장치와 통신하려면 실제 컴퓨터와 마찬가지로 네트워크 설정이 필요하다. 대표적으로 IP 주소, 서브넷 마스크, 게이트웨이, DNS 서버를 설정하게 되며, 이 정보들은 가상머신이 네트워크에서 자신의 위치를 식별하고 외부와 통신할 수 있도록 해준다.

1-1. IP 주소(IP Address)

IP 주소는 네트워크에서 장치를 식별하기 위한 고유한 주소이다. 사람이 집 주소를 통해 특정 집을 찾을 수 있는 것처럼, 컴퓨터는 IP 주소를 통해 특정 장치를 찾는다.

가상머신도 네트워크에 연결된 하나의 독립적인 컴퓨터로 취급되기 때문에 반드시 IP 주소를 가져야 한다.

예를 들어, VMware NAT 네트워크가 192.168.111.0/24 대역을 사용한다면 가상머신은 다음과 같은 주소를 가질 수 있다.

192.168.111.100

이 주소를 통해 다른 장치들은 해당 가상머신과 통신할 수 있다.

1-2. 서브넷 마스크(Subnet Mask)

서브넷 마스크는 IP 주소에서 어느 부분이 네트워크 주소이고 어느 부분이 호스트 주소인지를 구분하는 역할을 한다. 이를 통해 컴퓨터는 통신 대상이 같은 네트워크에 있는지, 다른 네트워크에 있는지를 판단한다.

예를 들어,

IP 주소 : 192.168.111.100
서브넷 마스크 : 255.255.255.0

이라면,

네트워크 주소 : 192.168.111.0
호스트 주소 : 100

으로 해석된다.

즉, 192.168.111.x 대역에 있는 장치들은 같은 네트워크에 속한다고 판단하게 된다.

1-3. 게이트웨이(Gateway)

게이트웨이는 다른 네트워크로 나가기 위한 출입구 역할을 하는 장치의 주소이다.

같은 네트워크에 있는 장치끼리는 직접 통신할 수 있지만, 인터넷이나 다른 네트워크로 나가려면 반드시 게이트웨이를 거쳐야 한다.

예를 들어, VMware NAT 환경에서는 VMware가 제공하는 가상 게이트웨이가 존재한다.

IP 주소 : 192.168.111.100 게이트웨이 : 192.168.111.1
--

가상머신이 인터넷에 접속하려고 하면 모든 패킷은 먼저 192.168.111.1 로 전달된다.

이후 VMware가 NAT(Network address Translation)을 수행하여 실제 인터넷으로 연결해준다.

1-4. DNS 서버(Domain Name System)

DNS 서버는 사람이 이해하기 쉬운 도메인 이름을 IP 주소로 변환해 주는 서버이다.

컴퓨터는 실제로 IP 주소를 사용하여 통신하지만, 사용자는 보통 도메인 이름을 사용한다.

예를 들어, 사용자가 웹 브라우저에서

www.google.com

을 입력하면 컴퓨터는 DNS 서버에 질의하여 해당 도메인의 IP 주소를 조회한다.

www.google.com ↓ DNS 서버 조회 ↓ 142.x.x.x ↓ 해당 서버 접속

DNS 서버가 설정되지 않으면 IP 주소로는 통신할 수 있지만 도메인 이름을 사용할 수 없게 된다.

1-5. VMware NAT 환경에서의 예시

VMware NAT 네트워크가 192.168.111.0/24 대역이라면 가상머신 설정은 다음과 같이 구성할 수 있다.

IP 주소	: 192.168.111.100
서브넷 마스크	: 255.255.255.0
게이트웨이	: 192.168.111.1
DNS 서버	: 8.8.8.8

이 경우 가상머신은 192.168.111.0/24 네트워크에 속하게 되며, 192.168.111.1을 통해 외부 인터넷으로 접속하고, DNS 서버를 통해 도메인 이름을 IP 주소로 변환하여 웹사이트에 접근할 수 있다.

결론적으로 IP 주소는 장치의 주소, 서브넷 마스크는 네트워크 범위 구분, 게이트웨이는 외부 네트워크로 나가는 출입구, DNS 서버는 도메인 이름을 IP 주소로 변환하는 역할을 수행한다. 이 네 가지 정보가 함께 설정되어야 가상머신이 정상적으로 네트워크를 사용할 수 있다.

2. NetworkManager와 네트워크 인터페이스 이해

2-1. nm-connection-editor란?

nm-connection-editor는 Linux에서 NetworkManager가 관리하는 네트워크 연결을 그래픽 환경에서 설정할 수 있는 도구이다. 터미널 명령어를 사용하지 않고도 IP 주소, 서브넷 마스크, 게이트웨이, DNS 서버 등의 네트워크 설정을 변경할 수 있도록 제공한다.

Linux에서는 NetworkManager가 네트워크 장치를 관리하는 경우가 많으며, nm-connection-editor는 이 NetworkManager의 설정을 편집하는 GUI 프로그램이라고 볼 수 있다. Windows의 “네트워크 어댑터 설정” 창과 비슷한 역할을 한다.

터미널에서 다음 명령을 실행하면 네트워크 설정 창이 열린다.

```
nm-connection-editor
```

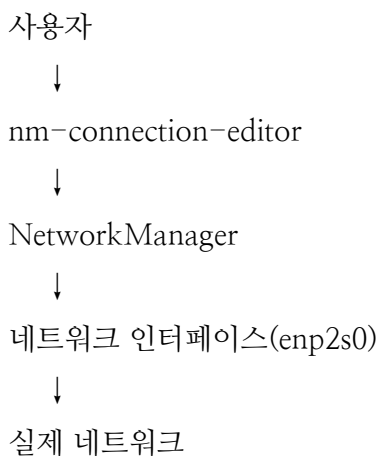
사용자는 이 창에서 현재 등록된 네트워크 연결을 확인하거나 새로운 연결을 생성할 수 있으며, 기존 연결의 IP 주소나 DNS 서버 등의 설정을 수정할 수도 있다.

2-2. NetworkManger란?

NetworkManager는 Linux에서 네트워크 연결을 관리하는 서비스이다. 유선 네트워크(Ethernet), 무선 네트워크(Wi-Fi), VPN 등의 연결 상태를 관리하며, 사용자가 네트워크를 설정하고 사용할 수 있도록 지원한다.

예전 Linux에서는 네트워크 설정 파일을 직접 수정해야 하는 경우가 많지만, 현재는 대부분 NetworkManager가 네트워크 설정을 관리한다.

구조를 단순하게 표현하면 다음과 같다.



즉, 사용자가 GUI를 통해 설정을 변경하려면 NetworkManager가 해당 설정을 실제 네트워크 장치에 적용한다.

2-3. Ethernet이란?

Ehternet은 유선 네트워크 연결을 의미한다. 일반적으로 가상머신이나 서버 환경에서는 무선 네트워크보다 Ethernet 연결을 주로 사용한다.

nm-connection-editor를 실행하면 다음과 같은 연결이 보일 수 있다.

Ethernet

여기서 Ethernet은 네트워크 연결 프로파일(Connection Profile)의 이름이다.

이 프로파일 안에는 다음과 같은 정보가 저장된다.

- IP 주소
- 서브넷 마스크
- 게이트웨이
- DNS 서버
- DHCP 사용 여부
- 네트워크 인터페이스 정보

즉, Ethernet은 실제 네트워크 카드가 아니라 해당 네트워크 카드에 적용할 설정 묶음이라고 이해하면 된다.

2-4. 네트워크 인터페이스란?

네트워크 인터페이스는 운영체제가 네트워크 장치를 식별하기 위해 사용하는 이름이다.

Windows에서는 보통

이더넷
Wi-Fi

와 같은 이름을 사용하지만,

Linux에서는 다음과 같은 형태의 이름을 사용한다.

enp2s0
ens33
enp0s3
eno1

이러한 이름은 실제 네트워크 카드 또는 가상 네트워크 어댑터를 의미한다.

2-5. enp2s0란?

enp2s0는 Linux가 네트워크 인터페이스에 부여한 이름이다.

과거 Linux에서는 네트워크 장치 이름이 다음과 같았다.

```
eth0  
eth1  
eth2
```

하지만 장치 순서가 바뀌면 이름도 바뀌는 문제가 있었다.

이를 해결하기 위해 현대 Linux에서는 Predictable Network Interface Names(예측 가능한 인터페이스 이름) 방식을 사용한다.

그래서 다음과 같은 이름이 사용된다.

```
enp2s0  
ens33  
enp0s3  
eno1
```

2-6. enp2s0 이름 분석

이름을 분석하면 다음과 같다.

```
en  = Ethernet 장치  
p2  = PCI 버스 2번  
s0  = 슬롯 0번
```

즉, enp2s0는 PCI 버스 2번 슬롯 0번에 연결된 Ethernet 네트워크 인터페이스라는 의미를 가진다.

사용자는 내부 구조를 외울 필요는 없고, “enp2s0 = 현재 사용 중인 유선 네트워크 카드” 정도로 이해하면 충분하다.

3. Linux 설치 미디어와 /dev/sr0의 의미

3-1. /dev/sr0란?

Linux를 설치하거나 사용하다 보면 다음과 같은 내용을 볼 수 있다.

```
/dev/sr0
```

또는

```
/dev/sr0 on /run/media/root/Rocky-10-0-aarch64-dvd type iso9660
```

여기서 /dev/sr0는 Linux가 인식한 CD/DVD 드라이브 장치를 의미한다. 과거에는 실제 CD-ROM이나 DVD-ROM 장치를 가리켰지만, 오늘날 가상머신에서는 대부분 ISO 파일이 연결된 가상 CD/DVD 드라이브를 의미한다.

예를 들어, VMware에서 Rocky Linux ISO 파일을 연결하면 Linux는 이를 실제 DVD가 삽입된 것처럼 인식한다.

```
Rocky-10.iso
```

↓

```
VMware 가상 DVD
```

↓

```
/dev/sr0
```

즉, /dev/sr0는 실제 DVD가 아니라 ISO 파일을 담고 있는 가상 DVD 장치라고 이해하면 된다.

3-2. iso9660 파일 시스템

다음과 같은 출력에서

```
/dev/sr0 on /run/media/root/Rocky-10-0-aarch64-dvd type iso9660
```

iso9660은 CD/DVD에서 사용하는 표준 파일 시스템이다.

즉, Linux는 현재 /dev/sr0에 삽입된 디스크를 읽어서

```
/run/media/root/Rocky-10-0-aarch64-dvd
```

위치에 마운트한 상태라는 의미이다.

쉽게 말하면

```
ISO 파일
↓
가상 DVD 삽입
↓
/dev/sr0
↓
마운트
↓
/run/media/root/Rocky-10-0-aarch64-dvd
```

가 된 것이다.

3-3. /BaseOS/Packages 폴더의 의미

Rocky Linux 설치 DVD 내부를 보면 다음과 같은 디렉터리가 존재한다.

```
BaseOS
AppStream
EFI
images
isolinux
```

그 중

```
/BaseOS/Packages
```

는 Rocky Linux를 구성하는 RPM 패키지들이 저장된 위치이다.

예를 들어, 내부에는

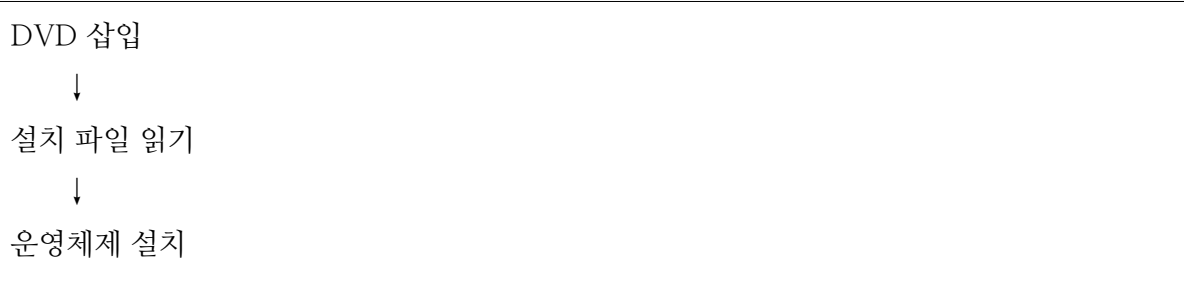
```
bash.rpm
vim.rpm
openssh.rpm
httpd.rpm
dnf.rpm
```

와 같은 파일들이 존재한다.

설치 프로그램은 운영체제 설치 과정에서 이 디렉터리의 RPM 패키지들을 읽어 시스템에 설치한다. 패키지 파일들이 알파벳 순으로 정렬되어 있는 것을 볼 수 있는데, 이는 단순히 저장소 관리 편의성을 위한 것이다.

3-4. CD/DVD를 넣고 빼는 것이 의미하는 것

과거 실제 서버에서는 다음과 같은 방식이었다.



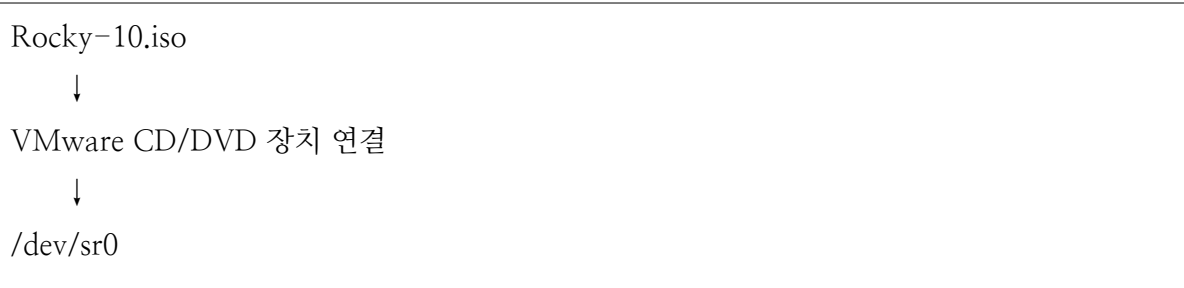
설치가 끝난 후에는 DVD 제거를 수행했다.

왜냐하면 부팅할 때마다 DVD로 부팅하지 않도록 하기 위해서이다.

3-5. 가상머신에서 DVD를 빼는 의미

VMware에서는 실제 DVD가 존재하지 않는다. 대신 ISO 파일이 연결되어 있다.

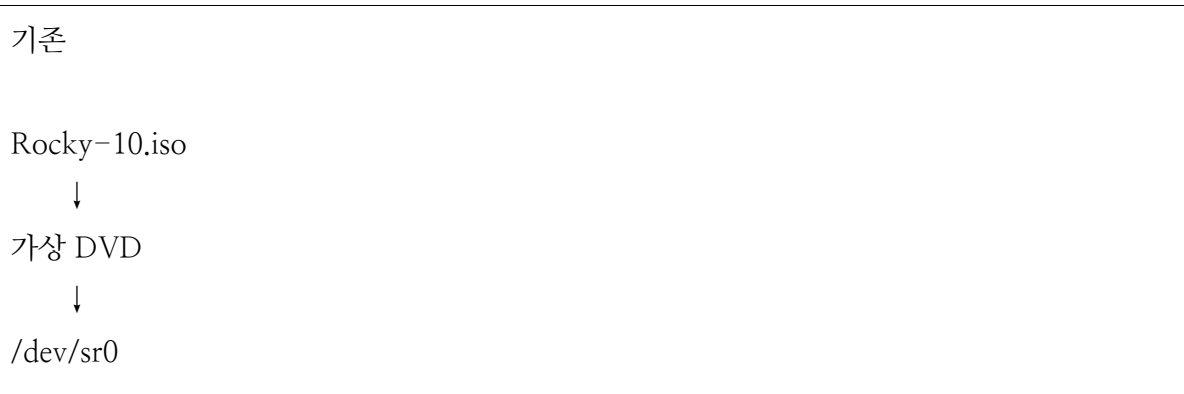
예를 들어,



상태이다.

여기서 “CD/DVD 제거”를 누른다는 것은 ISO 파일 연결 해제를 의미한다.

즉,



에서

제거 후

(연결 없음)



가상 DVD 비어 있음

상태가 된다.

3-5. 왜 ISO를 제거하는가?

설치가 끝난 후에도 ISO가 연결되어 있으면 VM이 재부팅될 때마다 설치 DVD가 삽입된 상태로 인식될 수 있다.

그러면 Rocky Linux 설치 화면이 다시 나타날 수도 있다.

따라서 설치가 완료되면 일반적으로 ISO를 제거한다.

운영체제 설치 완료



ISO 제거



가상 디스크로 부팅



정상 사용

과정을 거치게 된다.

4. Linux 사용자 생성과 /etc/skel의 역할

4-1. Linux의 사용자와 그룹 관리

Linux는 다중 사용자 운영체제이기 때문에 여러 사용자를 생성하여 동시에 사용할 수 있다. 관리자는 사용자와 그룹을 자유롭게 생성하고 삭제할 수 있으며, 각 사용자에게 독립적인 작업 환경을 제공할 수 있다.

그룹은 일반적인 groupadd 명령어를 사용하여 생성하고, 사용자는 useradd 명령어를 사용하여 생성한다. 또한 passwd 명령어를 통해 비밀번호를 설정할 수 있으며, 더 이상 필요하지 않은 사용자는 userdel -r 명령어를 사용하여 홈 디렉터리와 함께 삭제할 수 있다.

```
groupadd developers
useradd user1
passwd user1
userdel -r user1
```

4-2. 사용자 정보가 저장되는 위치

사용자를 생성하면 Linux는 관련 정보를 여러 설정 파일에 저장한다. 대표적으로 다음 파일들이 사용된다.

```
/etc/group 그룹보기
/etc/passwd 사용자보기
/etc/shadow 사용자 비밀번호 보기
```

이 파일들은 사용자 계정과 그룹을 관리하기 위한 핵심 데이터베이스 역할을 수행한다.

4-3. /etc/passwd 파일

/etc/passwd 파일에는 시스템에 존재하는 사용자 계정 정보가 저장된다. 예를 들어, 다음과 같은 형태로 저장된다.

```
user1:x:1001:1001:User One:/home/user1:/bin/bash
```

여기에는 다음과 같은 정보가 포함된다.

```
사용자 이름
UID(User ID)
기본 GID(Group ID)
홈 디렉터리
기본 로그인 셸
```

4-4. /etc/shadow 파일

/etc/shadow 파일에는 사용자 비밀번호 정보가 저장된다.

예를 들어, 다음과 같은 내용을 확인할 수 있다.

```
user1:$y$j9T$.....:20619:0:99999:7:::  
user2:$y$j9T$.....:20619:0:99999:7:::
```

비밀번호는 평문으로 저장되지 않고 암호화된 해시(Hash) 형태로 저장된다.

따라서 사용자를 생성하고 비밀번호를 설정하면 해당 정보가 /etc/shadow 파일에 기록된다.

```
tail -5 /etc/shadow
```

명령을 사용하면 최근 생성된 사용자 정보를 확인할 수 있다.

4-5. /etc/group 파일

/etc/group 파일에는 시스템의 그룹 정보가 저장된다.

예를 들어, 다음과 같은 형태를 볼 수 있다.

```
developers:x:1001:user1,user2
```

여기에는

```
그룹 이름  
GID(Group ID)  
그룹에 속한 사용자
```

정보가 저장된다.

즉, /etc/group은 “어떤 그룹이 존재하며 누가 해당 그룹에 속해 있는가”를 관리하는 파일이다.

4-6. 홈 디렉터리 생성

사용자를 생성하면 Linux는 일반적으로 /home 아래에 해당 사용자의 홈 디렉터를 생성한다. 예를 들어, user1을 생성하면 다음과 같은 구조가 만들어진다.

```
/home  
├── rocky  
├── user1  
└── user2
```

그리고 사용자 홈 디렉터리를 확인해보면 다음과 같은 기본 파일들이 존재한다.

```
/home/user1
├── .bash_logout
├── .bash_profile
├── .bashrc
└── .mozilla
```

흥미로운 점은 새로 생성한 사용자들이 거의 동일한 파일 구조를 가지고 있다는 것이다.

4-7. /etc/skel의 역할

새로운 사용자가 생성될 때 기본 파일들을 제공하는 템플릿 디렉터리가 바로 /etc/skel이다.
다음 명령으로 내용을 확인할 수 있다.

```
ls -a /etc/skel
```

예를 들어, 다음과 같은 파일들이 존재할 수 있다.

```
/etc/skel
├── .bash_logout
├── .bash_profile
├── .bashrc
└── .mozilla
```

사용자가 새로 생성하면 Linux는 /etc/skel 안의 내용을 새로운 사용자의 홈 디렉터리로 복사한다.

동작 과정은 다음과 같다.

```
/etc/skel
  ↓
useradd user1
  ↓
/home/user1
```

따라서 /home/user1 안에는 /etc/skel에 존재하던 파일들이 자동으로 복사된다.

4-8. /etc/skel을 사용하는 이유

/etc/skel의 가장 큰 장점은 모든 사용자에게 동일한 초기 환경을 제공할 수 있다는 점이다. 예를 들어, 서버를 운영하면서 모든 사용자에게 다음과 같은 항목을 제공하고 싶을 수 있다.

공통 .bashrc 설정
회사 안내 문서
기본 작업 디렉터리
스크립트 파일
환경 설정 파일

이러한 파일을 미리 /etc/skel에 넣어두면 이후 생성되는 모든 사용자에게 자동으로 복사된다.

관리자
↓
/etc/skel 수정
↓
새 사용자 생성
↓
동일한 환경 자동 제공

따라서 사용자마다 일일이 파일을 복사할 필요가 없으며, 다수의 계정을 관리하는 서버 환경에서 매우 효율적으로 사용할 수 있다.

4-9. 정리

Linux에서는 useradd, groupadd, passwd, userdel 명령어를 사용하여 사용자와 그룹을 관리한다. 사용자 정보는 /etc/passwd, 비밀번호 정보는 /etc/shadow, 그룹 정보는 /etc/group 파일에 저장된다.

사용자가 생성되면 /home 아래에 홈 디렉터리가 생성되며, 이때 /etc/skel 디렉터리의 내용이 자동으로 복사된다. 따라서 /etc/skel은 새로운 사용자의 기본 작업 환경을 제공하는 템플릿 디렉터리의 역할을 하며, 서버 환경에서는 여러 사용자에게 동일한 초기 설정을 제공할 때 매우 유용하게 사용된다.

5. 사용자 전환과 파일 권한 관리

5-1. 사용자 전환

Linux는 여러 사용자가 동시에 사용할 수 있도록 설계된 운영체제이다.

따라서 시스템 관리자는 필요에 따라 다른 사용자 계정으로 전환하여 작업할 수 있다.

사용자를 전환할 때는 su(Substitute User) 명령어를 사용한다.

예를 들어, 현재 root 계정에서 rocky 사용자로 전환하려면 다음과 같이 입력한다.

```
su - rocky
```

여기서 - 옵션은 매우 중요하다.

단순히

```
su rocky
```

를 실행하면 사용자만 변경되고 현재 환경 변수 일부가 유지될 수 있다.

반면

```
su - rocky
```

를 사용하면 rocky 사용자가 새롭게 로그인한 것과 동일한 환경이 구성된다.

현재 사용자

↓

```
su - rocky
```

↓

rocky 사용자 로그인 환경 적용

↓

홈 디렉터리 변경

환경 변수 변경

PATH 변경

따라서 일반적으로 사용자 전환 시에는 su - 사용자명 형태를 사용하는 것이 권장된다.

5-2. 파일 소유자와 그룹

Linux의 모든 파일과 디렉터리에는 소유자(Owner)와 그룹(Group)이 존재한다.

예를 들어, 다음과 같은 파일이 있다고 가정하자.

```
-rw-r--r-- 1 rocky developers 1024 Jun 16 testfile
```

여기서

rocky → 파일 소유자

developers → 파일 그룹

을 의미한다.

운영체제는 파일에 접근하는 사용자가 소유자인지, 같은 그룹에 속해 있는지, 또는 다른 사용자인지를 기준으로 권한을 판단한다.

5-3. Linux 파일 권한 구조

Linux의 파일 권한은 다음 세 종류의 사용자에게 대해 각각 설정된다.

User (소유자)

Group (그룹)

Others (기타 사용자)

각 사용자 유형에 대해 다음 세 가지 권한을 부여한다.

Read (읽기)

Write (쓰기)

Execute (실행)

파일 권한을 확인하면 다음과 같이 표시된다.

```
-rwxr-xr-x
```

이를 분리하면 다음과 같다.

```
-rwx | r-x | r-x
```

```
↓   ↓   ↓
```

```
User Group Others
```

5-4. 권한 숫자 표현 방식

Linux에서는 권한을 숫자로 표현하는 방법을 자주 사용한다.

기본 값은 다음과 같다.

```
4 = Read (읽기)
2 = Write (쓰기)
1 = Execute (실행)
```

권한 숫자를 더하여 표현한다.

```
7 = 4 + 2 + 1 = rwx
6 = 4 + 2    = rw-
5 = 4 + 1    = r-x
4 = 4        = r--
0 = 권한 없음 = ---
```

예를 들어,

```
chmod 755 testfile
```

다음 의미를 가진다.

```
소유자(User)    : rwx (7)
그룹(Group)     : r-x (5)
기타 사용자(Others): r-x (5)
```

결과적으로 파일 권한은 다음과 같이 표시된다.

```
-rwxr-xr-x
```

5-5. chmod 명령어

chmod(Change Mode)는 파일이나 디렉터리의 권한을 변경하는 명령어이다.

예를 들어, 다음 명령은 소유자에게 모든 권한을 부여하고, 그룹과 기타 사용자에게는 읽기와 실행 권한만 부여한다.

```
chmod 755 testfile
```

또는,

```
chmod 644 testfile
```

와 같이 사용할 수도 있다.

```
User  : rw-  
Group : r--  
Others: r--
```

5-6. chown 명령어

chown(Change Owner)은 파일의 소유자를 변경하는 명령어이다.

예를 들어, testfile의 소유자를 rocky 사용자로 변경하려면 다음과 같이 입력한다.

```
chown rocky testfile
```

소유자와 그룹을 동시에 변경할 수도 있다.

```
chown rocky:developers testfile
```

구조는 다음과 같다.

```
기존 소유자  
  ↓  
chown rocky testfile  
  ↓  
새 소유자 : rocky
```

소유자 변경은 시스템 보안과 관련된 작업이므로 일반적으로 root 사용자만 수행할 수 있다.

5-7. chgrp 명령어

chgrp(Change Group)는 파일의 그룹을 변경하는 명령어이다.

예를 들어, testfile의 그룹을 developers로 변경하려면 다음과 같이 입력한다.

```
chgrp developers testfile
```

구조는 다음과 같다.

기존 그룹

↓

```
chgrp developers testfile
```

↓

새 그룹 : developers

5-8. mv 명령어

mv(Move) 명령어는 파일의 이름을 변경하거나 다른 위치로 이동할 때 사용한다.

파일 이름을 변경하는 경우는 다음과 같다.

```
mv testfile rocky
```

testfile

↓

newfile

파일을 다른 디렉터리로 이동하는 경우는 다음과 같다.

```
mv testfile /home/rocky/
```

현재 위치

↓

```
mv testfile /home/rocky/
```

↓

/home/rocky/testfile

이 경우 파일 이름은 유지되고 저장 위치만 변경된다.

5-9. mv 정리

Linux는 다중 사용자 환경을 지원하기 때문에 사용자 관리와 파일 권한 관리가 매우 중요하다. 사용자는 `su - 사용자명` 명령어를 통해 다른 사용자로 전환할 수 있으며, 모든 파일과 디렉터리에는 소유자와 그룹이 존재한다. 파일 권한은 읽기(4), 쓰기(2), 실행(1)의 조합으로 표현되며, `chmod`는 권한 변경, `chown`은 소유자 변경, `chgrp`는 그룹 변경을 담당한다.

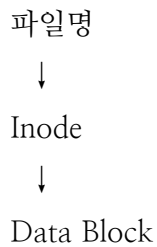
또한 `mv` 명령어를 이용하여 파일 이름 변경 및 위치 이동이 가능하다.

이러한 권한 관리 체계는 Linux 시스템의 보안과 다중 사용자 환경을 유지하는 핵심 기능이다.

6. Linux의 링크(Link)와 Inode

6-1. Linux 파일 시스템과 Inode

Linux에서 파일은 단순히 파일명으로 관리되지 않는다. 실제로는 파일명 → inode → Data block 구조로 저장된다.



여기서 Inode는 파일의 메타데이터를 저장하는 구조체이다. 파일 크기, 소유자, 권한, 생성 기간 등의 정보와 실제 데이터가 저장된 Data Block의 위치를 가지고 있다. 반면 파일명은 Inode 번호를 가리키는 역할만 수행한다.

즉, Linux에서 중요한 것은 파일명이 아니라 Inode라고 볼 수 있다.

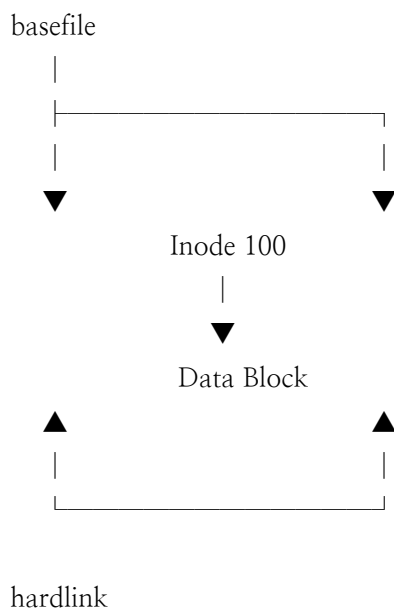
6-2. 하드 링크(Hard Link)

하드 링크는 새로운 파일을 만드는 것이 아니라 기존 파일의 Inode를 함께 공유하는 새로운 이름을 만드는 것이다.

예를 들어 다음과 같이 하드 링크를 생성할 수 있다.

ln basefile hardlink

생성 후 구조는 다음과 같다.



여기서 basefile과 hardlink는 서로 다른 파일명이지만 같은 Inode를 사용한다.
따라서 어느 쪽을 수정하든 실제 데이터가 변경되고, 다른 이름으로 접근해도 동일한 내용을 확인할 수 있다.

```
echo "hello" > basefile  
cat hardlink
```

를 실행하면 다음과 같은 결과가 나온다.

```
결과: hello
```

왜냐하면 두 파일명이 동일한 Inode와 Data Block을 공유하기 때문이다.

6-3. 하드 링크의 특징

하드 링크는 원본 파일과 완전히 동일한 파일로 취급된다.

따라서 원본 파일명을 삭제하더라도 다른 하드 링크가 남아 있으면 실제 데이터는 삭제되지 않는다.

예를 들어,

```
rm basefile  
cat hardlink
```

여전히 파일을 읽을 수 있다.

연결이 여전히 존재하기 때문에 실제로 Linux는 해당 Inode를 참조하는 링크 수가 0이 되었을 때만 데이터를 삭제한다.

6-4. 심볼릭 링크(Symbolic Link)

심볼릭 링크는 원본 파일의 Inode를 공유하지 않는다.

대신 새로운 Inode를 생성하고 원본 파일의 경로를 저장하는 특수한 파일을 만든다.

다음 명령으로 생성할 수 있다.

```
ln -s basefile softlink
```

생성 후 구조는 다음과 같다.

```
softlink
  ↓
Inode 200
  ↓
"basefile" 경로 저장

basefile
  ↓
Inode 100
  ↓
Data Block
```

즉, 심볼릭 링크는 실제 데이터를 가리키는 것이 아니라, 원본 파일의 위치를 기록한 바로가기 (Shortcut) 파일이라고 볼 수 있다.

Winodwos의 바로가기 아이콘과 매우 비슷한 개념이다.

6-5. 심볼릭 링크 사용

심볼릭 링크를 통해 파이레링 접근하면 운영체제가 먼저 원본 파일을 찾아서 데이터를 읽어온다.

예를 들어,

```
echo "hello" > basefile
cat softlink
```

를 실행하면

```
hello
```

가 출력된다.

사용자는 softlink를 읽고 있지만 실제 데이터는 basefile에서 읽고 있는 것이다.

6-6. 심볼릭 링크가 깨지는 이유

심볼릭 링크는 원본 파일의 경로를 문자열 형태로 취급한다.

예를 들어,

```
ln -s basefile softlink
```

를 생성한 경우

softlink

↓

"basefile"

라는 경로 정보만 저장된다.

그런데 원본 파일을 이동하면 문제가 발생한다.

```
mv basefile ../
```

실행 후 구조는 다음과 같다.

softlink

↓

"basefile"

현재 위치에는

basefile 없음

심볼릭 링크는 여전히 현재 디렉터리의 basefile을 찾으려고 하지만 파일이 다른 위치로 이동했기 때문에 더 이상 찾을 수 없게 된다.

이를 “링크가 깨졌다”고 표현한다.

```
cat softlink
```

를 실행하면 오류가 발생한다.

6-6. 정리

Linux 파일 시스템에서 실제 파일명은 파일명, Inode, Data Block 구조로 관리된다.

하드 링크는 기존 파일과 동일한 Inode를 공유하는 새로운 이름을 만드는 방식이며, 원본 파일을 삭제하더라도 다른 링크가 남아 있으면 데이터가 유지된다.

반면 심볼릭 링크는 새로운 Inode를 생성하여 원본 파일의 경로를 저장하는 특수한 파일로, Windwos의 바로가기와 유사한 개념이다.

따라서 원본 파일이 이동되거나 삭제되면 심볼릭 링크는 더 이상 원본을 찾을 수 없어 깨진 링크가 된다. Linux에서 링크를 이해하려면 “하드 링크는 Inode를 공유하고, 심볼릭 링크는 경로를 저장한다”

7. DNF의 동작 방식과 설정 파일

DNF는 Rocky Linux, RHEL, Fedora 계열에서 사용하는 패키지 관리자이다.

사용자가 `dnf install` 명령을 입력하면 단순히 패키지 하나를 다운로드하는 것이 아니라, 설정 파일을 읽고 저장소(Repository)와 통신하여 필요한 패키지와 의존성을 자동으로 설치하는 과정을 수행한다.

예를 들어, 다음 명령을 입력했다고 가정해보자.

```
dnf install httpd
```

사용자는 단순히 Apache 웹 서버를 설치하겠다는 명령만 내렸지만, DNF 내부에서는 여러 단계의 작업이 순차적으로 진행된다.

7-1. DNF Install 명령의 동작 과정

사용자가 “`dnf install 패키지명`”을 입력하면 가장 먼저 DNF는 저장소 정보가 기록된 설정 파일들을 확인한다.

```
사용자
  ↓
dnf install httpd
  ↓
/etc/yum.repos.d/*.repo 확인
```

여기에는 Rocky Linux 패키지 서버의 URL 정보가 저장되어 있다.

예를 들면, 다음과 같은 내용이 들어 있다.

```
[baseos]
name=Rocky Linux BaseOS
baseurl=https://download.rockylinux.org/...
enabled=1
gpgcheck=1
```

DNF는 이 URL을 이용하여 Rocky Linux 저장소에 접속한다.

저장소 주소를 확인한 후 DNF는 전체 패키지 목록(메타데이터)을 요청한다.

```
DNF
↓
Rocky Linux 저장소 접속
↓
패키지 목록(metadata) 요청
```

이 단계에서는 실제 RPM 파일을 다운로드하지 않는다.

단지

```
httpd
openssl
vim
bash
kernel
...
```

과 같은 패키지 목록 정보만 받아온다.

이후 DNF는 사용자가 요청한 패키지와 관련된 의존성(Dependency)을 분석한다.

예를 들어,

```
dnf install httpd
```

를 입력하면

```
httpd
httpd-core
mod_http2
apr
apr-util
...
```

등 함께 설치되어야 하는 패키지를 계산한다.

그리고 설치될 패키지 목록과 다운로드 용량을 화면에 출력한다.

Install 8 Packages

Total download size: XX MB

Installed size: XX MB

Is this ok [y/N]:

사용자가

Y

를 입력하면 그때 비로소 실제 RPM 패키지 파일을 다운로드한다.

DNF

↓

필요한 RPM 파일 요청

↓

패키지 다운로드

↓

자동 설치

즉, DNF는 처음부터 RPM 파일을 다운로드하는 것이 아니라, 먼저 저장소 메타데이터를 받아와서 어떤 패키지가 필요한지 분석한 후 실제 설치를 진행한다.

7-2. /etc/yum.conf 파일

/etc/yum.conf

파일은 DNF의 전반적인 동작 방식을 설정하는 파일이다.

예를 들어,

gpgcheck=1

installonly_limit=3

clean_requirements_on_remove=True

와 같은 설정들이 저장되어 있다.

실제 서버 운영에서는 기본값으로 사용하는 경우가 대부분이며 특별히 변경할 일은 많지 않다.

즉, DNF 전체 설정 파일이라고 이해하면 된다.

7-3. /etc/yum.repos.d 디렉터리

DNF에서 가장 중요한 설정 위치는 다음 디렉터리이다.

```
/etc/yum.repos.d/
```

이 디렉터리 안에는 여러 개의 .repo 파일이 존재한다.

예를 들면,

```
/etc/yum.repos.d/  
├── rocky.repo  
├── rocky-extras.repo  
├── rocky-addons.repo  
└── epel.repo
```

와 같은 형태이다.

이 파일들에는

```
어느 저장소에 접속할 것인가?  
어떤 URL을 사용할 것인가?  
해당 저장소를 사용할 것인가?
```

에 대한 정보가 저장되어 있다.

즉, DNF가 네트워크를 통해 패키지를 검색할 때 가장 먼저 참고하는 파일들이다.

7-4. *.repo 파일의 역할

.repo 파일은 DNF에게 패키지를 어디서 다운로드할지 알려주는 역할을 한다.

예를 들어,

```
[This10]  
name=Rocky Linux 10  
baseurl=http://192.168.111.100/This10  
enabled=1  
gpgcheck=0
```

와 같이 설정할 수 있다.

그러면 DNF는 Rocky Linux 공식 서버가 아니라 해당 URL에서 패키지를 검색하게 된다.

DNF
↓
.repo 파일 확인
↓
저장소 URL 확인
↓
패키지 검색 및 다운로드

과정으로 동작한다.

7-5. 실습 환경에서 This10.repo만 남겨두는 이유

실습에서는 Rocky Linux 10.0 패키지만 설치되도록 하기 위해

/etc/yum.repos.d/

안의 기존 저장소 설정을 모두 제거하고

This10.repo

하나만 남겨두는 경우가 있다.

구조는 다음과 같다.

/etc/yum.repos.d/
└── This10.repo

이렇게 하면 DNF는 다른 Rocky 저장소를 전혀 검색하지 않고 오직 This10.repo에 지정된 저장소만 사용한다.

그 결과

Rocky Linux 10.0 전용 패키지 설치
버전 고정
실습 환경 통일
업데이트 방지

와 같은 효과를 얻을 수 있다.

7-6. 정리

DNF는 `dnf install` 패키지명 명령이 실행되면 먼저 `/etc/yum.repos.d/` 디렉터리의 `.repo` 파일을 읽어 저장소 URL을 확인한다. 이후 저장소에서 전체 패키지 목록(메타데이터)을 다운로드 하고, 사용자가 요청한 패키지와 필요한 의존성을 계산하여 화면에 출력한다.

사용자가 설치를 승인하면 실제 RPM 패키지 파일을 다운로드하여 자동 설치한다.

`/etc/yum.conf`는 DNF의 전체 동작을 제어하는 설정 파일이며, `/etc/yum.repos.d/*.repo` 파일들은 패키지를 검색할 저장소의 주소를 정의한다.

실습 환경에서는 Rocky Linux 10.0만 사용하도록 기존 `.repo` 파일을 제거하고 `This10.repo` 하나만 남겨 특정 저장소만 사용하도록 구성하기도 한다.

8. Rocky Linux DVD를 저장소(Repository)로 사용하는 이유

일반적으로 DNF는 인터넷에 있는 Rocky Linux 저장소 서버에 접속하여 패키지를 다운로드한다. 하지만 Rocky Linux 설치 DVD(ISO) 안에는 운영체제 설치에 필요한 대부분의 패키지가 이미 포함되어 있다. 따라서 굳이 인터넷을 통해 다시 다운로드 하지 않고 DVD를 저장소로 활용할 수도 있다.

특히 네트워크 환경이 좋지 않거나 인터넷 비용이 비싼 환경에서는 DVD 저장소 방식이 매우 유용하다.

우리나라처럼 초고속 인터넷이 잘 구축된 환경에서는 인터넷 저장소를 사용하는 것이 당연하게 느껴질 수 있지만, 네트워크 인프라가 부족하거나 외부 인터넷 연결 비용이 높은 국가에서는 DVD 한 장에 포함된 패키지를 활용하는 것이 훨씬 경제적이 될 수 있다.

예를 들어, 기업 내부망이나 군사망처럼 인터넷 연결이 제한된 환경에서는 DVD 저장소를 사용하는 경우가 많다.

8-1. DVD를 저장소로 사용하기 위한 준비

먼저 DVD 장치를 마운트해야 한다.

```
umount /dev/cdrom
mkdir /media/cdrom
mount /dev/cdrom /media/cdrom
```

여기서 /dev/cdrom은 DVD 장치를 의미하며, 마운트가 완료되면 DVD 내부의 파일들을 /media/cdrom 에서 확인할 수 있다.

구조는 다음과 같다.

```
/dev/cdrom
  ↓
mount
  ↓
/media/cdrom
├── BaseOS
├── AppStream
├── EFI
└── images
```

즉, Rocky Linux ISO 안에 있던 내용이 그대로 보이게 된다.

8-2. 기존 네트워크 저장소 제거

DNF는 기본적으로 /etc/yum.repos.d/ 안의 .repo 파일을 참고한다.

실습에서는 기존 저장소 설정을 제거하기 위해 다음 작업을 수행했다.

```
cd /etc/yum.repos.d/  
mv This10.repo backup/
```

이렇게 하면 DNF는 더 이상 기존 네트워크 저장소를 사용하지 못한다.

이후 캐시를 삭제한다.

```
dnf clean all
```

이는 기존 저장소 정보를 모두 지우고 새로운 저장소 설정을 다시 읽도록 하기 위한 작업이다.

8-3. dvd.repo 생성

이제 DVD를 저장소로 사용하기 위한 설정 파일을 생성한다.

```
[dvd-baseos]  
name=Rocky Linux DVD BaseOS  
baseurl=file:///media/cdrom/BaseOS/  
gpgcheck=0  
  
[dvd-appstream]  
name=Rocky Linux DVD AppStream  
baseurl=file:///media/cdrom/AppStream/  
gpgcheck=0
```

여기서 가장 중요한 부분은 다음이다.

```
baseurl=file:///media/cdrom/BaseOS/
```

보통은

```
baseurl=https://...
```

형태로 인터넷 주소가 들어 있는데

```
file://
```

을 사용하면 네트워크가 아니라 로컬 파일 시스템을 사용한다.

즉, DNF는 인터넷 서버가 아닌 DVD 안의 패키지를 직접 읽게 된다.

구조는 다음과 같다.

```
DNF
↓
dvd.repo
↓
file:///media/cdrom/BaseOS
↓
DVD 내부 패키지
↓
설치
```

8-4. 네트워크를 끊고도 설치가 가능한 이유

실습 중에 다음 명령을 실행했다.

```
ifconfig enp2s0 down
```

이는 네트워크 인터페이스를 비활성화하는 명령이다.

즉 “인터넷 연결 없음” 상태가 된다.

실제로

```
ping -c 5 www.google.com
```

도 실패하게 된다.

그럼에도 불구하고 다음 명령은 정상적으로 동작한다.

```
dnf install mc
dnf install help2man
dnf install mariadb
```

그 이유는 DNF가 인터넷이 아니라 DVD 안에 있는 패키지를 읽고 있기 때문이다.

구조를 보면 이해하기 쉽다.

기존 방식

DNF



인터넷 저장소



패키지 다운로드

DVD 방식

DNF



DVD 저장소



패키지 설치

인터넷이 필요하지 않다.

8-5. network-crb 저장소의 의미

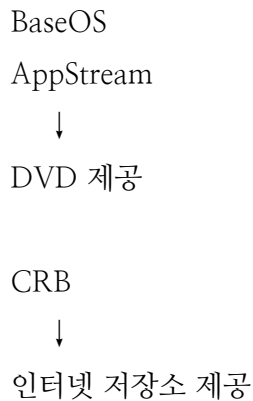
다만 DVD에는 모든 패키지가 포함되어 있는 것은 아니다.

그래서 다음과 같은 저장소를 추가하기도 한다.

```
[network-crb]
name=Rocky Linux Network CRB
baseurl=https://download.rockylinux.org/pub/rocky/10.0/CRB/aarch64/os/
        https://dl.rockylinux.org/vault/rocky/10.0/CRB/aarch64/os/
gpgcheck=0
```

CRB(CodeReady Builder)는 개발용 라이브러리나 추가 패키지가 포함된 저장소이다.

즉,



형태라고 생각하면 된다.

따라서 대부분의 패키지는 DVD에서 설치하고, DVD에 없는 일부 패키지만 인터넷 저장소를 통해 설치할 수 있다.

8-6. 정리

Rocky Linux 설치 DVD에는 운영체제 설치에 필요한 대부분의 RPM 패키지가 이미 포함되어 있다. 따라서 DNF가 인터넷 저장소에 접속하도록 하지 않고 DVD를 저장소로 설정하면 네트워크 없이도 패키지를 설치할 수 있다.

이를 위해 DVD를 /media/cdrom에 마운트하고, /etc/yum.repos.d/dvd.repo 파일을 생성하여 file:///media/cdrom/BaseOS와 file:///media/cdrom/AppStream을 저장소로 지정한다. 이렇게 하면 DNF는 인터넷 대신 DVD 안의 패키지를 직접 사용하게 되며, 네트워크 환경이 열악하거나 비용이 높은 환경에서는 훨씬 효율적인 설치 방법이 된다.

다만 DVD에 없는 일부 패키지는 CRB와 같은 인터넷 저장소를 추가로 사용해야 할 수도 있다.

9. DNF 그룹 패키지(Group Package) 관리

DNF는 개별 패키지뿐만 아니라 관련된 패키지들을 하나의 그룹으로 묶어서 관리하는 기능을 제공한다. 예를 들어, Java 개발 환경을 구축하려면 JDK, 컴파일러, 라이브러리 등 여러 패키지가 필요한데, 이를 하나씩 설치하는 대신 “Java Platform”이라는 그룹을 한 번에 설치할 수 있다.

이 기능은 서버 구축이나 개발 환경 구성 시 매우 편리하다. 사용자는 필요한 기능만 선택하면 되고, DNF가 관련 패키지들을 자동으로 설치해 준다.

9-1. DNF Group List 확인

시스템에 어떤 그룹이 존재하는지 확인하려면 다음 명령을 사용한다.

```
dnf grouplist
```

이 명령은 사용자가 설치할 수 있는 대표적인 그룹 목록을 출력한다.

예를 들면 다음과 같은 결과를 볼 수 있다.

Available Environment Groups:

Server

Minimal Install

Workstation

Available Groups:

Java Platform

Development Tools

Security Tools

여기서 그룹 하나가 여러 개의 패키지를 묶어 놓은 집합이라고 생각하면 된다.

9-2. 숨겨진 그룹 확인

일반적인 grouplist 명령은 모든 그룹을 보여주지 않는다.

숨겨진 그룹까지 확인하려면 다음 명령을 사용한다.

```
dnf grouplist hidden
```

이 명령을 실행하면 DNF 내부에서 사용하는 그룹까지 포함하여 모든 그룹이 출력된다.

즉,

```
dnf grouplist
```

↓

일반 그룹만 표시

```
dnf grouplist hidden
```

↓

일반 그룹 + 숨겨진 그룹 표시

의 차이가 있다.

실습에서는 전체 그룹 목록을 파일로 저장하기 위해 다음과 같은 명령을 사용할 수 있다.

```
dnf grouplist hidden > glist.txt
```

이 명령은 출력 결과를 화면에 보여주는 대신 glist.txt 파일에 저장한다.

결과적으로

```
dnf grouplist hidden
```

↓

```
glist.txt
```

형태가 된다.

이후에는

```
cat glist.txt
```

로 내용을 확인할 수 있다.

9-3. DNF 캐시 삭제

DNF는 패키지 목록과 저장소 정보를 캐시에 저장하여 속도를 높인다.

하지만 저장소 설정을 변경했거나 최신 정보를 다시 받아와야 하는 경우에는 캐시를 삭제해야 한다.

이를 위해 사용하는 명령이 다음과 같다.

```
dnf clean all
```

실행하면

저장소 메타데이터 삭제
패키지 캐시 삭제

가 수행된다.

이후 DNF를 실행하면 저장소로부터 최신 정보를 다시 받아온다.

즉,

```
dnf clean all
```

↓

기존 캐시 제거

↓

다음 dnf 실행 시 재다운로드

과정이 진행된다.

9-4. Java Platform 그룹 설치

특정 그룹을 설치하려면 groupinstall 명령을 사용한다.

예를 들어, Java 개발 환경 전체를 설치하려면 다음과 같이 입력한다.

```
dnf -y groupinstall "Java Platform"
```

실행 과정은 다음과 같다.

Java Platform 그룹 선택

↓

관련 패키지 목록 확인

↓

OpenJDK

컴파일러

라이브러리
개발 도구
↓
자동 설치

즉, 사용자는 Java Platform 그룹 하나만 지정하지만 실제로는 수십 개의 관련된 패키지가 함께 설치될 수 있다.

9-5. 텍스트 모드 전환(chvt 3)

Linux는 여러 개의 가상 콘솔(Virtual Terminal)을 제공한다.

다음 명령을 실행하면

```
chvt 3
```

3번 가상 콘솔로 이동한다.

보통 그래픽 환경은

```
tty1 또는 tty2
```

를 사용하고,

```
tty3  
tty4  
tty5  
...
```

는 텍스트 모드 콘솔로 사용된다.

즉,

GUI 화면
↓
chvt 3
↓
텍스트 콘솔(tty3)

형태로 전환된다.

서버 관리나 문제 해결 시 GUI 없이 작업해야 할 때 자주 사용된다